

Structured Belief State and the First Precision-Aware Benchmark for LLM Memory Retrieval

Jeffrey Flynt
Independent Researcher
jeffrey.flynt@utexas.edu

May 2026

Abstract

Every major benchmark for LLM memory systems, LoCoMo foremost, measures whether a model answered correctly, not whether the memory system retrieved correctly. A system returning its entire belief store achieves recall of 1.0 and passes answer-quality evaluation. This is the difference between a unit test and an integration test: retrieval quality must be measured in isolation from the generative model it feeds into, and no existing benchmark does this.

We demonstrate that this failure persists even when entity extraction is entirely faithful. Memory baselines achieve mean retrieval precision of just 0.05 to 0.08 on cases referencing their own extractions. The failure is structural: cosine similarity over a domain-specific corpus cannot discriminate relevant beliefs from semantically proximate ones, an invariance confirmed across a 20x range in embedding model scale. Multi-turn evaluation surfaces a compounding failure; after topic drift, comparison systems allow semantic mass to bleed across turns, yielding high drift scores on re-entry. Single-turn metrics conceal this cost: Hindsight reports sub-700ms single-turn latency but exceeds 2,700ms mean per session turn, with p95 above 6,000ms. Under LLM-as-a-Judge evaluation, these failures remain invisible.

We present two contributions: PrecisionMemBench, an 89-case benchmark measuring retrieval precision independently of generative models across diverse scope, mutation, and isolation assertions; and Tenure, a local-first structured belief store using multi-path BM25 with analyzer asymmetry, differential boosting, and hard scope isolation. Tenure passes 89/89 cases with mean precision 1.0 and sub-15ms retrieval latency. Comparison providers perform worse than the raw vector baseline they are built on, with zero active retrieval passes and ingestion costs of 98 to 897 seconds, failures that answer-quality benchmarks cannot detect.

Code: github.com/tenurehq/tenure Benchmark: github.com/tenurehq/precisionmembench

1 Introduction

1.1 The Evaluation Gap

Memory systems for LLM agents have a measurement problem. Every major benchmark used to evaluate them, LoCoMo [4] foremost, measures whether a model answered correctly given retrieved context. None measures whether the memory system retrieved correctly. These are not the same

question, and treating them as interchangeable has allowed a systematic retrieval failure to go undetected across the field.

The failure mode is straightforward. A memory system that returns its entire belief store achieves recall of 1.0 trivially. At the corpus sizes current benchmarks operate on (tens to low hundreds of beliefs), a capable generative model can locate the correct answer in a noisy context window and score well on F1, BLEU, or LLM-as-a-Judge metrics. The result is that systems with mean retrieval precision of 0.05 to 0.08 appear competitive with systems that retrieve exactly the right beliefs and nothing else.

Strip the generative model out and route retrieved context directly to a classifier, rules engine, or any structured pipeline: current retrieval implementations fail immediately, not because the right belief is absent, but because it is surrounded by enough noise that a non-generative consumer cannot act on it. The generative model was never a neutral downstream consumer; it was load-bearing infrastructure compensating for retrieval imprecision.

1.2 The Re-Orientation Tax

The underlying problem memory systems are trying to solve is real. Every LLM session begins from a blank context window. A developer who established in a prior session that their project uses TypeScript with strict mode, Fastify for HTTP, and MongoDB with the raw driver must re-establish all of this before the model can produce useful output. The correction prompt is not progress; it is overhead that compounds across every session. The dominant response, retrieval-augmented memory implemented by ChatGPT memory [7], Mem0 [2], Memori [1], and similar systems, treats cross-session memory as a search problem. The evaluation methodology the field has adopted is structurally incapable of detecting the failure this causes, because it routes retrieved context through a generative model before measuring anything.

1.3 Why Similarity Search Fails

In any belief store where participants share a technical domain, all beliefs about that domain occupy a common semantic region. Cosine similarity captures this domain proximity but cannot discriminate within it. A query about Redis is semantically close to a belief about Redis as intended and also semantically close to beliefs about MongoDB, TypeScript, Fastify, Kubernetes, and GitHub Actions, with cosine scores between 0.65 and 0.83. The scores reflect genuine semantic relatedness. They are measuring the wrong thing. A larger embedding model distributes scores differently but cannot eliminate genuine semantic proximity within a domain-specific corpus. Resolving it requires a retrieval signal orthogonal to semantic similarity.

1.4 Contributions

1. **A precision-aware retrieval evaluation benchmark** (Section 6): 89 cases covering alias resolution, scope disambiguation, supersession chain exclusion, fuzzy matching, cross-user isolation, budget eviction, ranking stability, and session-level noise isolation under multi-turn topic drift. Cases carry `mustExclude` assertions and `shouldOnlyInclude` constraints that make noise a hard failure rather than an invisible inference cost. The benchmark evaluates retrieval quality independently of any generative model and is published as a reusable artifact against which any memory implementation can be measured.
2. **A formal belief schema** (Section 3) with five belief types, four epistemic statuses, typed

provenance, and a `why_it_matters` field that converts extracted facts into pre-computed action instructions rather than raw material for the model to re-derive.

3. **A precision-first retrieval design** (Section 4): multi-path BM25 with index-search analyzer asymmetry, differential boosting, and hard scope isolation, grounded in the named entity resolution argument that alias-weighted term matching provides higher precision than semantic similarity as a structural consequence in bounded vocabulary contexts where the query author and belief author share terminology. The empirical results are in Section 6.
4. **A compaction architecture** (Section 5) that prevents noise floor accumulation over time, including an alias enrichment flywheel and a supersession mechanic that structurally retires outdated context rather than relying on retrieval to suppress it.

2 Related Work

2.1 The Benchmark Gap

No existing benchmark measures retrieval precision as a first-class signal for LLM memory systems. LoCoMo [4], LongMemEval [3], and the evaluations built on top of them measure answer quality through a generative judge. This design makes them structurally unable to distinguish a memory system that retrieves one correct belief from one that retrieves the entire store.

LoCoMo was designed to evaluate model behavior over long-term conversations: question answering, event summarization, and multimodal dialogue generation. The unit of analysis is the model’s comprehension of lengthy context, not the memory system’s retrieval precision. The field adopted it as a memory system benchmark without examining whether it measures the right properties for that purpose.

The scaling failure this conceals is predictable. At tens of beliefs, a model can sort noise. At thousands of beliefs (the scale any serious persistent memory use case reaches), full-corpus retrieval becomes architecturally impossible and the precision problem can no longer be offloaded to inference. No current benchmark operates at this scale. LoCoMo comprises 10 conversations [4], and LongMemEval does not report retrieval precision as a separate metric [3].

2.2 The Incomplete Fix in Recent Memory Systems

The field has correctly diagnosed the disease. Some have then prescribed the right treatment and administered it wrong. Others never left the pathogen. Write-time structure is the right treatment; embedding similarity at read time reintroduces the noise that structure was designed to eliminate, and a system that never reaches structured extraction has not treated anything.

Mem0 [2] commits to write-time extraction: facts are extracted from conversation turns and stored as natural language strings rather than raw transcripts. This is architecturally correct. Mem0 then retrieves at read time using embedding similarity, which reintroduces the noise that structured extraction was designed to eliminate.

Memori [1] is the closest published system to Tenure’s architectural position. Both reject raw transcript injection, treat memory as a structuring problem, and extract structured representations at write time. Memori converts dialogue into semantic triples (subject-predicate-object) linked to conversation summaries, and retrieves using a hybrid of cosine similarity and BM25 over triple content, an acknowledgment that keyword matching is necessary even when embeddings are available.

The distinction is in representation: Memori’s triples are declarative records of facts; Tenure’s beliefs are imperative instructions for how future responses should use those facts. That difference is developed in Section 3.

A-MEM [13] treats memory as a self-organizing network of Zettelkasten-inspired notes where new memories trigger updates to the contextual representations of existing ones. This evolution mechanism inherits the context rot problem by design: there is no supersession chain, no audit log, and no structural guarantee that outdated context is retired.

2.3 Structured Memory Evaluation

StructMemEval [11] evaluates an agent’s ability to organize long-term memory rather than merely retrieve facts, isolating tasks requiring state tracking, hierarchical organization, and accumulated counting. Its central finding maps directly onto the argument this paper makes: retrieval-augmented systems fail these tasks even when the underlying facts are available and the retrieval budget is generous. What it does not measure is whether the memory system retrieved the right beliefs in the first place. StructMemEval inherits the same evaluation gap as LoCoMo: correctness is assessed at the answer layer, not the retrieval layer, leaving a system that returns the entire store indistinguishable from one that returns exactly what was needed.

2.4 Dual-Route Retrieval and Structural Discrimination

H-Mem [14] and Mnemis [12] independently arrive at the same architectural conclusion: a single retrieval signal is insufficient, and a second discriminating pass is necessary after initial retrieval. Both systems respond to this by adding infrastructure: temporal and semantic parallel trees in H-Mem, a System-2 Global Selection mechanism over a hierarchical graph in Mnemis. The pattern across recent high-performing memory systems is convergent: more retrieval paths, heavier embedding infrastructure, and re-ranking stages added on top of similarity search. Tenure’s position is that the additional infrastructure is compensating for the wrong primary signal. Alias-weighted BM25 with hard scope isolation requires no re-ranking stage because the first-pass result is already precise.

2.5 Named Entity Resolution and BM25

The choice of BM25 [10] over embedding similarity is grounded in the named entity resolution literature [5]. For single-user knowledge stores where the user coined the terminology and aliases were authored to match expected query surfaces, BM25 with alias boosting provides higher precision than cosine similarity by construction. Embedding similarity retrieves what is semantically related; alias-weighted BM25 retrieves what the user named. In a single-user persistent memory context, the latter is more often correct. The linguistic basis for this claim, documented in the idiolect and lexical priming literature, is developed in Section 4.

3 The Belief Architecture

3.1 Overview

A *belief* is the atomic unit of persistent context. The typed belief schema with epistemic status is a practical descendant of the Belief layer in the Belief-Desire-Intention (BDI) agent architecture [9]. BDI formalizes beliefs as the agent’s informational state (what it holds to be true about the world),

distinct from its goal state (Desires) and committed action plans (Intentions). Tenure inherits this representational commitment: the belief store is the informational substrate the model reasons from, not a search index it queries. The departure from BDI is scope: Tenure does not model Desires or Intentions, because the LLM session provides those dynamically. What persists across sessions is the belief state alone.

Beliefs are not conversation fragments; they are extracted conclusions, typed, scoped, and versioned, that can be injected into future sessions without requiring the model to re-derive them from raw history. The belief store controls all factual state; the model renders surface responses from it. StructMemEval [11] provides independent empirical motivation: tasks requiring state tracking and hierarchical organization are reliably solved by structured memory agents but not by retrieval-augmented systems.

The contrast with Memori’s semantic triples [1] is precise: a triple records a fact; a belief records a fact, its epistemic status, its scope, its provenance, and its supersession history, along with an imperative instruction for how a future model should act on it. The additional structure is the mechanism that prevents context rot rather than merely describing it.

Definition 1 (Belief). *A belief b is a 14-field tuple with the following typed components:*

- *$type \in \{\text{preference}, \text{decision}, \text{entity}, \text{open_question}, \text{relation}\}$*
- *$subtype \in \{\text{expertise}, \text{style}, \text{null}\}$*
- *$epistemic_status \in \{\text{active}, \text{inferred}, \text{exploratory}, \text{superseded}\}$*
- *$scope \subseteq \mathcal{S}$, the set of valid scope labels*
- *$confidence \in [0, 1]$*
- *$superseded_by \in \{id'\} \cup \{\text{null}\}$*

A belief is *active* if `superseded_by = null` and `resolved_at = null`.

3.2 The `why_it_matters` Field as Pre-Computed Action

The `why_it_matters` field is the architectural feature that most sharply distinguishes a belief from a semantic triple or a flat extracted fact. Every belief requires it: if the extracting model cannot state what future responses this fact shapes, the belief is not extracted.

The distinction is between declarative and imperative representation. “Uses TypeScript with strict mode” is a fact. “Shapes all code examples toward TypeScript with strict mode and no implicit any” is an instruction. A model receiving the fact must still infer how to apply it. A model receiving the instruction acts on it directly without additional inference work. The imperative form is what makes structured extraction qualitatively better than transcript injection, not merely token-efficient.

The `why_it_matters` field is why extraction timing matters independently of token cost. The extracting model writes `why_it_matters` when it has the full conversational context and the complete reasoning chain present. The right inference is captured at the moment of its formation.

Persona Prelude

Above individual beliefs, a persona prelude generated from the accumulated belief state is injected unconditionally on every session. It provides standing behavioral instruction covering communication

style, working preferences, and ambiguity handling, addressing the cold-start problem independently of query-driven retrieval.

3.3 Belief Types

Preferences represent how the user works and communicates, shaping model behavior without requiring query matching.

Decisions represent resolved commitments including the rejected alternatives, so settled questions are not re-litigated in future sessions.

Entities represent named things whose identity must survive surface variation in how the user refers to them.

Open questions represent unresolved matters that future sessions should surface rather than answer.

Expertise beliefs (a subtype of preference) represent depth calibration: what the user already knows, so the model engages at the right level without being told.

3.4 Epistemic Status

Epistemic status is a first-class field rather than a derived property of confidence. The **active** versus **inferred** distinction determines injection behavior and the threshold for promotion across sessions. The **active** versus **superseded** distinction is the architectural mechanism that prevents context rot.

Superseded beliefs are retained in the store for audit purposes but are never injected into any session. A system that deletes stale beliefs cannot distinguish between “we never had this belief” and “we had this belief and moved past it.” This is the structural property that neither Memori’s semantic triples [1] nor Mem0’s flat memory graph [2] provide.

3.5 Scope

Scope assigns each belief to a context boundary: universal beliefs inject across all sessions, domain beliefs inject only within a matching domain, and project beliefs inject only within a matching project. Scope is a hard filter: beliefs outside the active scope are structurally absent from retrieval rather than probabilistically suppressed. A session in one project cannot surface beliefs from another regardless of semantic proximity.

4 Retrieval Design

4.1 The Precision-First Argument

Write-time extraction is architecturally correct; similarity search at read time reintroduces the noise that structured extraction was designed to eliminate. The precision-first argument is the architectural answer: wherever aliases are authored to match expected query surfaces, term matching over indexed aliases provides higher precision than semantic similarity by construction.

The simplest case is a single user who coined the terminology: if they named their Kubernetes belief with canonical name **kubernetes** and aliases **k8s** and **kube**, then a query containing **k8s** should retrieve that belief with high precision regardless of semantic distance. There is no ambiguity to

resolve: the authored terminology is the ground truth. The same property holds for an engineering team whose shared codebase and runbooks have converged on consistent vocabulary.

The predictable objection is that BM25 fails on vocabulary the system has not yet indexed. This misidentifies a correct behavior as a limitation. Think of how a co-worker introduces a term you have never heard: you do not pretend to recognize it, and you do not guess at something adjacent. You note it, and from that point forward you have it. That is exactly how the system behaves. On first encounter it returns silence rather than noise. The extraction worker captures the new term as an alias. Every subsequent query using that term resolves correctly. Real memory works this way. The alias enrichment flywheel that implements this acquisition cycle is described in Section 5.

The Semantic Noise Tax. Vector retrieval advocates often point to zero-shot synonym matching (e.g., matching *login* to *authentication*) as the primary advantage of cosine similarity. We argue this is a false economy. The price of zero-shot synonym matching is the inability to isolate distinct concepts that share a semantic region. As the session-drift evaluation demonstrates (Section 6.4), a vector database that catches a synonym on re-entry does so while dragging the semantic mass of preceding off-topic turns into the context window. Tenure trades the temporary friction of a cold-start synonym miss for a permanent, structural guarantee against context bleed. The alias enrichment flywheel (Section 5) resolves the cold-start miss organically through use; the noise tax, once paid, cannot be recovered.

This design exploits a documented property of individual language production. Corpus-based idiolect research establishes that single speakers maintain stable, distinctive lexical choices across production contexts, with idiolectal patterns consistent over periods of one to two years [15]. Lexical priming theory formalizes the mechanism [16]: words become entrained through use, and speakers reliably return to the same lexical choices in the same topical contexts [17]. A single-user belief store is precisely the setting where these properties are strongest: the query author and the belief author are the same person. The property extends to any context where vocabulary has converged, including engineering teams with shared codebases and terminology.

4.2 Search Architecture and Index Engineering

The Inverted Retrieval Problem Standard BM25 assumes short queries against long documents. Belief retrieval inverts this: queries are potentially hundreds of tokens of natural language while documents are short (a canonical name of one to three tokens and an alias list of three to five terms). Standard BM25 applied naively to this inverted setting fails because query term frequency becomes noise, IDF weights are computed over a tiny corpus of short identifiers, and standard analyzers are calibrated for prose rather than proper nouns. In practice, BM25 is abandoned at this first step. The index engineering that makes it work for belief retrieval is never reached.

The solution is domain-specific index engineering for the inverted problem. The canonical name field is analyzed to produce discrete matchable tokens from structured identifiers. A shingle analyzer produces phrase-level match surfaces from short alias lists. A high boost weight on canonical name and alias phrase matches re-calibrates the score distribution so that a single precise match on a short field outweighs accumulated term frequency noise from a long query. Search-side analyzer asymmetry ensures the query is tokenized conventionally while the index side is structured for precision against short fields.

Index-Search Analyzer Asymmetry Standard BM25 implementations apply the same analyzer to both the index and the query. For belief retrieval this is the wrong default. The index side needs to decompose structured identifiers into matchable tokens: a canonical name like `redis_cache` must enter the index as two discrete tokens so a natural language query containing `redis` resolves it. The query side needs conventional tokenization so that prose queries are not over-processed against a short-field index. These are different requirements, and satisfying both requires independent analyzer configuration at each side.

Three levels of asymmetry are required. First, canonical name fields must be indexed with a tokenizer that treats delimiters as token boundaries, while the query side uses standard prose tokenization. Without this, canonical name matching fails on any natural language query because structured identifiers are not naturally occurring phrases.

Second, alias fields require exact-surface matching without stemming. Stemming collapses distinct aliases that should remain independent match surfaces. The index side preserves surface forms; the query side applies only normalization.

Third, multi-word aliases require phrase-level match surfaces on the index side paired with phrase generation on the query side. A query containing *base class hierarchy* should match a belief whose alias set includes *base class* as a phrase, not just as two independent tokens that happen to co-occur.

This three-level asymmetry is not a property of any particular search platform. It is a structural requirement that follows from the mismatch between natural language queries and short structured identifiers. Any BM25 implementation that permits independent analyzer configuration at the index and query boundary can implement it.

Retrieval Pipeline Query text is cleaned before matching: code blocks, URLs, stack traces, and markup are stripped so that structured artifacts do not dominate term frequency.

Matching runs over two paths with differential boost weights: canonical name and alias phrases. Fuzzy matching with a prefix guard handles transpositions and near-miss terms while blocking prefix mismatches that edit distance alone would permit.

Scope, user identity, supersession status, and resolution status are applied as hard filters evaluated concurrently with scoring. A superseded or out-of-scope belief is never a candidate regardless of match quality. Pinned beliefs and open questions are retrieved by independent paths that exclude them from text search scoring entirely; they are also applied as hard filters so they cannot be weighted or ranked against retrieved candidates.

Pinned facts and type routing. Pinned beliefs and all active preferences are retrieved unconditionally, independent of text search scoring. Open questions are retrieved by a separate path that returns only pinned open questions for the active scope and are never returned by text search. Type routing takes precedence over both pin status and retrieval score.

5 Conflict Resolution and Compaction

5.1 Conflict Resolution

When a new extraction arrives, the merger asks one question: does this belief already exist, and if so, does the new version agree with it?

If the belief is new, it enters the store. If it matches an existing belief, the existing one is reinforced. If the two versions conflict, the merger does not silently overwrite: it queues the conflict for user review, preserving both versions until the user resolves it. If the conflict lacks sufficient confidence to act on at all, it is skipped.

Supersession is handled as a first-class event rather than an overwrite. When a user moves from one tool or preference to another, the old belief is marked as superseded and a pointer connects it to the replacement. The chain is preserved for audit. The old belief never injects again, but the record that it once existed does not disappear.

5.2 Compaction

The belief store grows monotonically as new beliefs are extracted. Without compaction, the store accumulates duplicate, overlapping, and redundant beliefs that degrade retrieval precision and inflate context size. Compaction is the architectural answer: it prevents noise floor accumulation rather than relying on retrieval to suppress it after the fact.

5.3 Alias Enrichment as a Precision Flywheel

The predictable objection to BM25 for belief retrieval is vocabulary coverage: if a user refers to a belief using a term not yet in the alias set, retrieval fails. This objection is correct as a static description and wrong as a practical one. The alias set is not static. It grows with every session, capturing the vocabulary the user actually uses as they use it.

Three properties make this work. First, every session is an observation of how the user refers to their beliefs in natural language. New surface forms are captured continuously and added to the alias set of the belief they describe. The system does not require the user to declare their vocabulary upfront; it learns it from use.

Second, vocabulary is never lost to compaction. When two beliefs merge into a single canonical form, every term that previously retrieved either belief continues to retrieve the merged one. The alias set accumulates across the full history of the store.

Third, the alias set includes counter-signals: terms for things the user has moved away from or explicitly rejected. A user who switched from one tool to another does not need to remember which tool they now use. A query referencing the old tool surfaces the belief about the new one. The system surfaces what the user actually does when a query references what they used to do.

The consequence is a precision flywheel that runs in the opposite direction from the objection. A purely semantic system degrades as the store grows: more beliefs means more semantic mass, broader cosine overlap, and lower precision on every query. Alias-weighted BM25 improves as the store grows: more sessions means more observed surface forms, a richer alias set, and higher precision on the vocabulary that is actually used. The store becomes more findable with each session, not less.

6 Evaluation

6.1 Benchmark Design

The benchmark evaluates retrieval quality independently of any generative model. Cases carry `mustExclude` assertions and `shouldOnlyInclude` constraints that make noise a hard failure rather

than an invisible inference cost. A system that returns every belief in the store achieves recall of 1.0 and fails every precision assertion. A system that returns nothing fails every recall assertion. Neither failure requires a downstream model to surface it.

Cases are evaluated against a fixed seed corpus of 35 beliefs spanning two domain scopes, a three-hop supersession chain, and a secondary-user fixture for isolation validation. Session cases extend the corpus dynamically: beliefs are created and alias sets are updated mid-session, validating that retrieval reflects the live store state rather than a snapshot taken at evaluation start.

Five configurations are evaluated against the benchmark: Tenure, a vector baseline (mxbai-embed-large, cosine similarity), Mem0 [2], Zep [8], and Hindsight [6]. All embedding-dependent configurations use **mxbai-embed-large** (1024 dimensions), ensuring performance differences reflect architecture rather than embedding model choice.

Providers requiring an LLM at read or write time were evaluated using Claude Sonnet 4.6, a more capable model than any provider used in their own published evaluations. The precision failures documented in Section 6.5 occur with the strongest available model in the extraction and retrieval path. All comparison systems were evaluated against pinned Docker image digests: Mem0 (memohq/mem0@sha256:276964b172d2), Zep (zep-zep@sha256:f9e5372caf60), and Hindsight (ghcr.io/vectorize-io/hindsight@sha256:f0f9e9a73d6a) were pulled and run without modification. Results reflect the published architecture at time of evaluation; later versions incorporating additional structural features (e.g., scope filtering or reranking) are not represented here and can be measured directly against the published benchmark.

Metrics

Four metrics are recorded per case. **Retrieval precision** and **retrieval recall** are computed over the **relevantBeliefs** tier on cases where that tier carries an active assertion. **Pinned coverage** is recorded on cases where the **pinnedFacts** tier is asserted. **Question precision** and **question recall** are recorded on cases where the **openQuestions** tier is asserted. Cases where a metric is structurally inapplicable (empty query, budget-forced empty, no relevant-belief assertion) record **null** for that metric and are excluded from aggregate computation for that metric only.

A *pass* requires all asserted tiers to be simultaneously satisfied. A case with **retrievalPrecision: 1.0** that also carries an unmet **pinnedCoverage** assertion fails.

Pass Taxonomy

Because comparison systems interact with the benchmark in structurally different ways, we distinguish three pass types in all results tables:

- A case passes **on retrieval** when it carries a **retrievalPrecision** assertion and that assertion is satisfied.
- A case passes **on structure** when it asserts scope isolation, supersession chain exclusion, type routing, or budget behavior without a retrieval precision assertion, and the structural property holds.
- A case passes **trivially** when the expected **relevantBeliefs** tier is empty by case design (empty query, **maxBeliefs: 0**, budget set to exact pinned count): any system returning an empty set passes by construction, and **retrievalPrecision** is **null**.

A system can accumulate passes exclusively in the trivial category while achieving zero retrieval passes. Aggregate pass counts without this breakdown obscure that failure.

6.2 Case Categories

Table 1: Evaluation benchmark case distribution by category.

Category	Case count
Alias resolution	23
Scope disambiguation	12
Fuzzy matching and prefix guards	8
Type routing and open questions	6
Budget eviction and capacity	5
Design boundary cases	6
Supersession chain exclusion	3
Relation expansion	4
Persona prelude content	4
Counter-signal retrieval	2
Ranking stability	3
Cross-user isolation	1
Cold start behavior	1
Session-level noise isolation	12
Total	89

Alias resolution (23 cases). Whether variant surface forms of a belief’s name: short-form, natural-language, and multi-word resolve to the correct belief.

Scope disambiguation (12 cases). Whether scope alone correctly discriminates between beliefs sharing an alias across different domain scopes.

Supersession chain exclusion (3 cases). Whether superseded beliefs are excluded at depth in a multi-hop chain. A query matching both a superseded and a superseding term must surface neither superseded belief; the active terminal belief surfaces via the pinned facts tier.

Fuzzy matching and prefix guards (7 cases). Whether the retrieval layer correctly handles transpositions and near-miss terms while blocking prefix mismatches that edit distance alone would permit. Both pass and fail behaviors are documented as intentional design properties.

Counter-signal retrieval (2 cases). Whether a query referencing a rejected or superseded term surfaces the active replacement belief via a counter-signal alias. Both cases carry an active retrieval precision assertion.

Relation expansion (4 cases). Whether relation-type beliefs correctly surface and expand their participants via a one-hop join, with participant type routing and scope filters applied during expansion.

Budget eviction and capacity (5 cases). Whether the retrieval layer handles slot constraints correctly, including graceful empty returns, single-slot priority, and resistance to high-reinforcement flooding at the budget ceiling.

Cross-user isolation and cold start (2 cases). Whether beliefs belonging to a second user are structurally excluded from a primary user’s retrieval regardless of semantic proximity, and whether a new user with zero seeded beliefs returns a fully empty context without error.

Session-level noise isolation (12 cases). Described in Section 6.4.

Infrastructure Footprint. The five systems differ substantially in deployment requirements. Tenure runs as a single Docker container on localhost. Zep [8] requires a multi-container deployment including a graph store and embedding pipeline, with ingestion latency of 897 seconds across the 35-belief seed corpus (mean 25,630ms per belief). Hindsight’s [6] full image bundles a BGE embedder (~130 MB) and a MiniLM cross-encoder (~90 MB) into memory with PyTorch/ONNX runtime overhead; a realistic full-stack deployment requires 3–4 GB RAM before session load. The cross-encoder reranker is the architectural feature that most distinguishes Hindsight’s [6] full image from a standard embedding retrieval stack, and its effect on precision is directly observable in the benchmark results.

Schema-Aware Evaluation Harness. A predictable objection to structured benchmarks is that they unfairly penalize traditional vector systems for lacking native routing logic. To prevent this, our evaluation harness grants all comparison systems a *schema-aware wrapper*. When evaluating any provider, the harness automatically applies pin-status filtering, open-question routing, and scope isolation to the provider’s retrieved IDs using Tenure’s own structural metadata. Consequently, comparison systems do not fail the benchmark due to formatting or structural technicalities; they fail purely because cosine similarity cannot prevent catastrophic noise accumulation (mean precision 0.05 to 0.08 across comparison systems), returning 10 to 18 irrelevant beliefs per query. The evaluation is, if anything, more favorable to comparison systems than to Tenure: they receive structural routing logic for free at evaluation time, while Tenure must implement and maintain that routing natively as part of its own retrieval pipeline.

Embedding Model Invariance

A natural objection to the vector baseline results is that a more capable embedding model would close the precision gap. To test this, we ran the benchmark against three embedding models spanning a wide range of scale and architecture: `nomic-embed-text` (768 dimensions), `mxbai-embed-large` (1024 dimensions), and `qwen3` (4096 dimensions) at the 8-billion parameter scale. All three use cosine similarity with no additional pipeline. As Table 2 shows, mean retrieval precision is 0.09 across all three models and pass counts are identical at 11/77.

The `qwen3` result is the clearest demonstration that this is not a capability problem. At roughly 20 times the mean latency of `nomic-embed-text` and over 1,100ms mean per query, the 8-billion parameter model produces identical precision. A more powerful embedder distributes scores differently but cannot eliminate genuine semantic proximity within a domain-specific corpus. The fix is not a better ruler; it is a different measurement instrument.

Table 2: Vector retrieval results across three embedding models. Precision and pass counts are invariant to model scale. All 11 passes in every configuration are trivially empty or budget-forced cases; active retrieval passes are 0 across all three models.

Model	Precision	Recall	Passes	Mean (ms)	p95 (ms)
nomic-embed-text (768)	0.09	1.0	11/77	43.36	85.21
mxbai-embed-large (1024)	0.09	1.0	11/77	96.48	257.24
qwen3-8b (4096)	0.09	1.0	11/77	1130.95	2604.84

6.3 Retrieval Results

Table 3: Retrieval results by case category across all evaluated systems. Active retrieval pass counts exclude trivially empty and budget-forced cases.

Category	Cases	Tenure	Vector	Mem0	Zep	Hindsight
Alias resolution	23	23/23	0/23	0/23	0/23	0/23
Scope disambiguation	12	10/12	1/12	0/12	0/12	0/12
Fuzzy matching	7	7/7	0/7	0/7	0/7	0/7
Relation expansion	4	4/4	0/4	0/4	0/4	0/4
Type routing / open questions	6	6/6	0/6	0/6	0/6	0/6
Budget eviction and capacity	5	5/5	3/5	3/5	3/5	3/5
Design boundary cases	6	6/6	2/6	2/6	2/6	2/6
Supersession chain exclusion	3	3/3	0/3	0/3	0/3	0/3
Persona prelude content	4	4/4	2/4	1/4	1/4	1/4
Counter-signal retrieval	2	2/2	0/2	0/2	0/2	0/2
Ranking stability	3	3/3	1/3	1/3	1/3	1/3
Cross-user isolation	1	1/1	1/1	1/1	1/1	1/1
Cold start behavior	1	1/1	1/1	1/1	1/1	1/1
Session-level noise	12	12/12	0/12	0/12	0/12	0/12
Total	89	89/89	11/89	9/89	9/89	8/89

Pass Type Breakdown

Aggregate pass counts require the pass taxonomy introduced in Section 6 to be interpreted correctly.

Retrieval Precision on Active-Assertion Cases

Mean retrieval precision and recall restricted to cases where `retrievalPrecision` is not `null`. Recall of 1.0 with precision below 1.0 is the indiscriminable full-corpus retrieval pattern.

Retrieval Latency

Retrieval latency is measured across all 77 cases per system. Ingestion latency is reported separately as it reflects write-time architecture rather than read-time performance.

Table 4: Pass type breakdown. Active retrieval passes require a `retrievalPrecision` assertion to be satisfied. Trivially empty and budget-forced passes require only that the `relevantBeliefs` tier be empty when the case design makes that expectation structurally necessary. Session-level noise isolation cases (12 of 89 total) are excluded from this table; they carry no `retrievalPrecision` assertion and are evaluated separately in Section 6.4. All counts are over the 77 non-session cases.

System	Active retrieval	Structural	Trivially empty
Tenure	48	21	8
Vector	0	8	3
Mem0	0	8	1
Zep	0	8	1
Hindsight	0	8	1

Table 5: Mean retrieval precision and recall on active-assertion cases only. Recall of 1.0 with precision below 1.0 is the indiscriminate full-corpus retrieval pattern.

System	Mean precision	Mean recall
Tenure	1.0	1.0
Vector	0.09	1.0
Mem0	0.05	0.99
Zep	0.08	0.95
Hindsight	0.05	1.0

Table 6: Retrieval latency (ms) per system. Ingestion latency reflects write-time extraction cost over the 35-belief seed corpus.

System	Mean (ms)	p50 (ms)	p95 (ms)	Ingestion total (s)
Tenure	13.49	9.77	53.99	0.98
Vector	96.48	71.87	257.24	—
Mem0	78.81	64.94	156.89	114.19
Zep	139.64	124.36	235.04	897.04
Hindsight	672.15	589.86	1,185.33	173.28

6.4 Session-Level Noise Isolation

The 12 session cases test a single orthogonal property: whether beliefs introduced during off-topic drift turns contaminate retrieval on subsequent unrelated turns. The primary case is a 10-turn session in which a topic is established at turn 0, followed by 8 drift turns across unrelated domains, followed by an implicit return to the original topic at turn 9. Per-turn noise-check assertions verify that drift-turn beliefs do not contaminate re-entry retrieval. The *drift score* is the fraction of retrieved non-pinned beliefs originating from drift-turn topics; 0 is perfect isolation.

Table 7: Session-level drift scores per system on noise-critical turns. Drift score is the fraction of retrieved non-pinned beliefs originating from drift-turn topics; 0 is perfect isolation.

Turn	Tenure	Vector	Mem0	Zep	Hindsight
Turn 9 (implicit re-entry)	0.0	1.0	1.0	1.0	1.0
Turn 10 (explicit re-entry)	0.0	0.94	1.0	1.0	0.94
Cross-session formative	0.0	0.94	1.0	0.92	1.0

At turn 10 the vector baseline ranks **b-redis-code** first (cosine 0.853) while simultaneously retrieving all 7 noise beliefs (drift score 0.50). Zep [8] achieves drift score 0.92 on the same turn, surfacing noise beliefs introduced during unrelated drift turns while retaining **b-redis-code** in the result set. Hindsight achieves drift score 1.0 at turn 10 with **b-redis-code** absent entirely (recall 0): the correct belief is not merely ranked low but missing from the result set, with 4 explicit noise belief violations flagged on top of the full-corpus return.

Hindsight’s published evaluation interposes a 20B model between retrieval and the chat backbone [6]. The precision failures documented here are not inputs to the chat model; they are inputs to that 20B intermediary, whose ability to recover from noisy retrieval is precisely what LoCoMo measures and what the benchmark is structurally incapable of distinguishing from correct retrieval.

Session-Turn Latency. Table 8 reports mean and p95 retrieval latency per session turn across the 12 session cases. All three comparison systems degrade under session load: Mem0 and Zep increase 3–4x over their single-turn means, while Hindsight, reporting 672ms mean single-turn latency, exceeds 2,700ms mean per session turn with p95 latency climbing above 6,000ms.

Table 8: Session-turn retrieval latency (ms) across the 12 session cases. Single-turn mean from Table 5 included for comparison. ST = single-turn mean; SM = session mean; SP50 = session p50; SP95 = session p95.

System	ST (ms)	SM (ms)	SP50 (ms)	SP95 (ms)
Tenure	13.49	49.02	47.79	134.63
Vector	96.48	319.17	256.75	727.68
Mem0	78.81	382.62	377.93	692.35
Zep	139.64	396.89	418.13	659.99
Hindsight	672.15	2,735.84	1,880.60	6,162.57

6.5 Failure Analysis

Extraction Quality Does Not Predict Retrieval Precision: A Relation-Type Case

The preference-belief case establishes that extraction quality and retrieval precision are decoupled. A relation-type belief makes the same argument more concretely, because relation beliefs have a structural property that preference beliefs do not: they carry participant pointers that a precision-aware system can follow deterministically.

Consider **b-auth-depends-redis**, a relation-type belief in the evaluation seed corpus:

```
canonical_name : auth_service_depends_on_redis
```

```

aliases      : ["auth redis", "session backend", "auth"]
content      : "Auth service depends on Redis for session
               storage.
               If Redis is down, auth fails open (denies all).
               why_it_matters : "Shapes failure-mode analysis auth and Redis
               are
               coupled; cannot discuss auth resilience without
               addressing Redis availability."
participants : ["b-redis-code", "b-auth-open-q"]

```

After ingestion with full `content` and `why_it_matters` fields, Mem0’s extraction pipeline produced the following stored memory, confirmed by direct database inspection:

```

"User’s auth service depends on Redis for session storage. If
Redis
goes down, auth fails open by denying all requests. Auth
resilience
discussions must address Redis availability the two are tightly
coupled."

```

The extraction preserved every operationally significant fact: the service name, the dependency target, the fail-open behavior, and the coupling assertion. This is a high-quality extraction by any measure.

The benchmark includes two cases that reference this belief directly. The first, `relation-type-expands-participants`, issues the query *“what are the auth service dependencies and failure modes?”* The expected result is two beliefs: `b-auth-depends-redis` (matched via alias) and `b-redis-code` (reached via one-hop participant join). Mem0 returns `b-auth-depends-redis` correctly, then returns 16 additional beliefs including `b-linting-v0`, `b-react-expertise`, `b-vitest-pref`, `b-comm-pushback`, and `b-sqlalchemy-superseded`. Retrieval precision is 0.056. `b-redis-code` is absent entirely (recall 0.5): the structurally necessary participant is missing from the result set despite being directly referenced in the stored memory text.

The second case, `relation-participant-open-question-routes-correctly`, issues the query *“session backend dependency graph.”* Here Mem0 achieves recall of 1.0: both `b-auth-depends-redis` and `b-redis-code` are present. Retrieval precision is 0.111: 16 additional beliefs were returned alongside the two correct ones, including `b-kubernetes-entity`, `b-functional-pipeline`, `b-sqlalchemy-superseded`, and `b-chapter-structure-q`, the last of which is a writing-domain open question with no relationship to the auth service at all.

The two cases together isolate the failure mode precisely. When the query is specific enough that cosine similarity locates both participants, it also locates 16 semantically proximate beliefs with no relevance to the query. When the query is slightly less specific, one participant disappears from the result set entirely. Neither outcome is caused by poor extraction. The stored memory text is accurate and complete.

Tenure’s indexed representation of the same belief is:

```

auth_service_depends_on_redis auth redis session backend auth

```

Four tokens. The `content` and `why_it_matters` fields are not indexed. The participant expansion is executed as a deterministic one-hop join on the `participants` array rather than as a semantic

search. Both cases pass with precision 1.0: the correct two beliefs are returned, the 16 noise beliefs are absent, and the participant routing via `b-auth-open-q` to `openQuestions` is enforced structurally rather than inferred from content similarity.

The relation-type case sharpens the earlier argument in one direction: the precision failure is not merely an artifact of soft preference language occupying a shared semantic region. It occurs on a concrete, operationally specific relation belief whose extraction was faithful and complete. And it occurs in both the high-recall and low-recall conditions, indicating that the precision floor is structural rather than query-dependent.

Full-Corpus Return

On alias resolution cases (a query using the alias `k8s` to retrieve the Kubernetes entity belief, or a scope disambiguation query for the Redis datastore belief in code scope), Hindsight [6] returns 18 beliefs with precision 0 and recall 0: the correct belief does not appear at all. This is not a ranking failure; it is a retrieval failure. The cross-encoder reranker bundled in Hindsight’s full image is the architectural feature designed to address exactly this class of problem. The benchmark results show it does not close the gap.

The latency cost of this architecture is substantial. Hindsight’s [6] mean single-turn retrieval latency is 672.15ms (p50 589.86ms, p95 1,185.33ms), with an ingestion cost of 173.3 seconds across 35 beliefs (mean 4,951ms per belief). Zep’s [8] mean single-turn latency of 139.64ms reflects a graph-based retrieval path without on-device model inference; its ingestion cost of 897.0 seconds across 35 beliefs (mean 25,630ms per belief) reflects write-time graph construction overhead.

Zep’s [8] async ingestion architecture that enables this low read-time latency creates a structural availability gap in fast conversational use: individual beliefs reach 125,148ms during ingestion, and at a mean of 25,630ms per belief against a conversational turn cadence of 10 to 30 seconds, a belief introduced at turn 1 may not be queryable until the session has largely concluded. The benchmark’s pre-ingestion design does not penalize this; all beliefs are confirmed available before any retrieval case runs. The 897-second total ingestion figure is therefore a lower bound on the availability gap in practice, not an artifact of evaluation methodology.

Session-Level Drift

After 8 consecutive off-topic drift turns, a return to the original topic produces drift scores of 0.94 to 1.0 across all three comparison systems (Table 7). The same structural argument applies: semantic mass accumulated during drift turns is indiscriminable from the re-entry topic, and neither richer ingestion input nor a more capable extraction pipeline changes the cosine geometry the retrieval layer operates on.

7 Discussion

User-controlled scope makes the state boundary explicit and auditable. Precision-first retrieval and hard scope isolation together provide a structural guarantee: the right beliefs surface, and only within the boundaries the user has authorized. In explicit scope mode this extends to a data governance property relevant to team deployments: a session in `project:client-a` cannot surface beliefs from `project:client-b` regardless of semantic proximity, because scope is a hard filter applied before retrieval scoring. A user moving between contexts mid-session receives an

immediate belief context pivot with no residual injection from the previous scope. These properties are architectural consequences of treating memory as typed state rather than a search index, and no surveyed system provides an equivalent guarantee.

The Missing Baseline. Neither the Mem0 ECAI evaluation [2] nor the Memori [1] LoCoMo evaluation [4] report a no-memory control condition. The retrieval results here make that gap empirically concrete. Mem0’s [2] failure mode is recall 1.0 with mean precision 0.08: the system returns all twelve beliefs on every active query regardless of relevance. A model evaluated against ECAI with Mem0 [2] attached would be sorting through eleven noise beliefs to find the relevant one on every turn. This failure is invisible to answer quality metrics as long as the downstream model is capable enough to compensate.

Zep and Hindsight. Zep [8] achieves 0 active retrieval passes with mean precision 0.06 and drift score 0.92 on the re-entry turn, surfacing noise beliefs from unrelated drift turns while retaining the correct belief in the result set at low rank. Hindsight [6] achieves 0 active retrieval passes with mean precision 0.05 and drift score 1.0 on the re-entry turn, with the correct belief absent entirely. A system marketed as state-of-the-art on agent memory [6] achieves zero active retrieval passes on the benchmark most directly measuring that claim, at 672ms mean single-turn latency and 2,736ms mean session-turn latency. The evaluation gap the paper identifies is not a gap in a legacy system; it is present in the most recently published work in the field.

The Non-Generating Consumer Argument The answer quality conflation matters beyond benchmark design. Any downstream consumer that is not a capable generative model exposes retrieval noise that answer quality metrics hide. A structured data pipeline, a classifier, a rules engine, or a fine-tuning dataset cannot compensate for a retrieval system that returns twelve beliefs when one was relevant. The “model will figure it out” safety net is a property of frontier generative models, not a property of retrieval. A memory system whose correctness depends on a generative model’s ability to reason under noise is architecturally coupled to that model’s inference capability. This coupling is invisible in answer quality benchmarks precisely because those benchmarks are evaluated using capable generative models.

The benchmark asserts against retrieved beliefs directly, independent of any downstream consumer, making the precision guarantee verifiable without a generative model in the evaluation loop.

8 Conclusion

We have presented Tenure, a local-first belief architecture for persistent cross-session context in LLM interactions. The central claim is that cross-session LLM memory is a state management problem, not a search problem, and that similarity search is the wrong retrieval paradigm for named entity resolution within bounded vocabulary contexts, of which a single user is the simplest instance.

Together, the four contributions form a single coherent answer to the re-orientation tax: a schema that makes belief state auditable, a retrieval design that makes injection precise, a compaction architecture that keeps the store clean over time, and a benchmark that makes all three properties measurable by any comparison system. The quantitative anchor is precision. Comparison systems achieve mean precision of 0.05 to 0.08 on active-assertion cases. The vector baseline passes 11/89 total cases, but all 11 are structural or trivially empty passes; it accumulates zero active retrieval passes out of the 48 cases that carry a `retrievalPrecision` assertion. Mem0, Zep, and Hindsight

each pass fewer total cases than the vector baseline they are built on, also with zero active retrieval passes. Tenure passes 89/89 with mean precision 1.0. Under multi-turn topic drift the gap widens: comparison systems produce drift scores of 0.92 to 1.0 on noise-critical turns where Tenure maintains 0.0 across all session turns.

The benchmark provides 89 cases against which any comparison system can be measured directly. Tenure itself ships as a Docker container and produces the results described here from the first session.

The re-orientation tax motivating the system remains an observational claim rather than a measured one. Retrieval precision is a necessary precondition for response quality improvement, not a sufficient one; that measurement is the open question this work makes tractable.

References

- [1] Borro, L. C., Macarini, L. A. B., Tindall, G., Montero, M., and Struck, A. B. (2026). Memori: A persistent memory layer for efficient, context-aware LLM agents. *arXiv preprint arXiv:2603.19935*.
- [2] Chhikara, P., Khant, D., Aryan, S., Singh, T., and Yadav, D. (2025). Mem0: Building production-ready AI agents with scalable long-term memory. In *Proceedings of the 27th European Conference on Artificial Intelligence (ECAI 2025)*. arXiv:2504.19413.
- [3] Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., and Yu, D. (2025). LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*.
- [4] Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., and Fang, Y. (2024). Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*.
- [5] Mihalcea, R. and Csomai, A. (2007). Wikify!: Linking documents to encyclopedic knowledge. In *Proceedings of the Sixteenth ACM Conference on Information and Knowledge Management, CIKM '07*, pages 233–242.
- [6] Latimer, C., Boschi, N., Neeser, A., Bartholomew, C., Srivastava, G., Wang, X., and Ramakrishnan, N. (2025). Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint arXiv:2512.12818*.
- [7] OpenAI (2023). Memory and new controls for ChatGPT. Retrieved from <https://openai.com/blog/memory-and-new-controls-for-chatgpt> (accessed May 2026).
- [8] Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., and Chalef, D. (2025). Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*.
- [9] Rao, A. S. and Georgeff, M. P. (1995). BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*, pages 312–319.
- [10] Robertson, S. E. and Walker, S. (1994). Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval. In *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '94*, pages 232–241.
- [11] Shutova, A., Olenina, A., Vinogradov, I., and Sinitsin, A. (2026). Evaluating memory structure in LLM agents. *arXiv preprint arXiv:2602.11243*.

- [12] Tang, Z., Yu, X., Xiao, Z., Wen, Z., Li, Z., Zhou, J., Wang, H., Wang, H., Huang, H., Deng, D., Sun, F., and Zhang, Q. (2026). Mnemis: Dual-route retrieval on hierarchical graphs for long-term LLM memory. *arXiv preprint arXiv:2602.15313*.
- [13] Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. (2025). A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*.
- [14] Ye, Z., Huang, J., Chen, W., and Zhang, Y. (2026). H-Mem: Hybrid multi-dimensional memory management for long-context conversational agents. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2026)*, pages 7756–7775.
- [15] Barlow, M. (2013). Individual usage: A corpus-based study of idiolects. *International Journal of Corpus Linguistics*, 18(4), 443–478.
- [16] Hoey, M. (2005). *Lexical Priming: A New Theory of Words and Language*. Routledge.
- [17] Wright, D. (2018). Idiolect. In M. Aronoff (Ed.), *Oxford Research Encyclopedia of Linguistics*. Oxford University Press.

A Belief Schema Reference

Table 9: Belief field reference.

Field	Type	Description
<code>_id</code>	string	UUID
<code>user_id</code>	string	Owner identifier
<code>type</code>	enum	preference, decision, entity, open_question, relation
<code>subtype</code>	enum	expertise, style, null
<code>canonical_name</code>	string	Snake_case primary identifier
<code>aliases</code>	string[]	Alternative terms including counter-signals; 3–5 at extraction, hard ceiling 25 after enrichment
<code>content</code>	string	The belief statement
<code>why_it_matters</code>	string	Imperative: what future responses this shapes
<code>scope</code>	string[]	Scope labels: user:universal, domain:X, or project:X
<code>epistemic_status</code>	enum	active, inferred, exploratory, superseded
<code>confidence</code>	float	[0, 1]
<code>pinned</code>	bool	Always inject regardless of query
<code>user_edited</code>	bool	User manually authored or corrected
<code>superseded_by</code>	string?	ID of replacement belief
<code>resolved_at</code>	date?	Timestamp of open question resolution
<code>reinforcement_count</code>	int	Number of reinforcement events
<code>last_reinforced_at</code>	date	Timestamp of most recent reinforcement
<code>provenance</code>	object	session_id, turn_id, extracted_at, source_model
<code>change_log</code>	object[]	Append-only mutation history
<code>expertise_domain</code>	string?	Hierarchical domain, e.g. javascript/react
<code>expertise_depth</code>	enum?	learning, working, deep, expert
<code>expertise_evidence_count</code>	int?	Source belief count for synthesis

Fields reflect the schema present in the Section 6 evaluation (v1.0, May 2026). The authoritative and current field reference is maintained at github.com/tenurehq/tenure.